



RELAZIONE MOLTIPLICATORE BINARIO 16 BIT UNSIGNED

A CURA DI:

MARIAIDA PERRI – MAT.239943

ANDREEA ROXANA MANOLACHE – MAT.245906

ANDREA MAZZA – MAT.240059

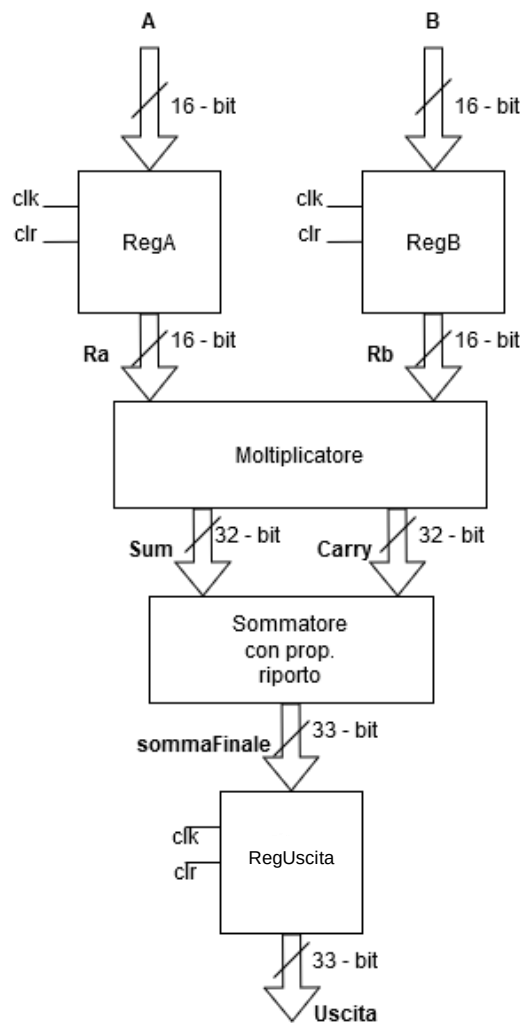
FRANCESCO PIO RUFFO – MAT.240044

A.A. 2024/25

ARCHITETTURA DI RIFERIMENTO E SCELTE PROGETTUALI

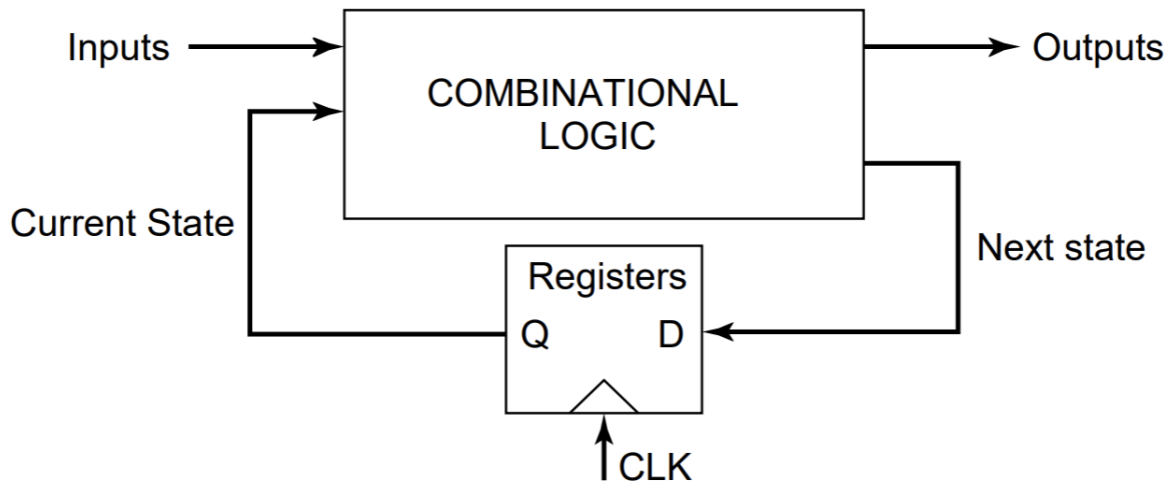
Per l'implementazione del moltiplicatore binario si è proceduto optando per una descrizione mista, seguendo una descrizione behavioral e structural.

Questa scelta consente di suddividere il progetto in componenti riutilizzabili e indipendenti tra di loro, favorendo il debug e la manutenibilità generale del codice stesso.



Come si evince dall'architettura di riferimento sovrariportata, al fine di sincronizzare gli ingressi A e B si è reso necessario l'uso dei registri; il progetto si basa dunque sull'unione di **componenti sequenziali** e **combinatorie**.

Un circuito sequenziale, a differenza di un circuito combinatorio, ha memoria degli eventi passati: il valore prodotto in uscita al tempo t dipende dagli ingressi ricevuti al tempo t e dall'uscita generata al tempo $t-1$.



Per tenere traccia del *prima-ora-dopo*, in questo tipo di circuiti c'è bisogno di inserire un *clock*, un orologio.

Il clock è caratterizzato da due eventi:

1. Livello: che a sua volta può essere:
 - Livello 0;
 - Livello 1;
2. Fronte: che a sua volta può essere:
 - Fronte di salita;
 - Fronte di discesa;

Questi due eventi sono due modi diversi con cui si può scandire il tempo.

Esempi di circuiti sequenziali sono i LATCH e i FLIP-FLOP.

Un LATCH è un circuito sensibile ai livelli del clock, un FLIP-FLOP è sensibile ai fronti del clock.

Un LATCH può essere:

- Positivo: trasferisce il dato in uscita quando il clock è uguale ad 1 e memorizza lo stato precedente quando il clock è uguale a zero;
- Negativo: trasferisce il dato in uscita quando il clock è uguale a 0 e memorizza lo stato precedente quando il clock è uguale ad 1.

Un FLIP-FLOP sensibile ai fronti di salita del clock trasferisce in uscita il dato quando il clock “sale” e memorizza lo stato precedente in tutti gli altri casi, mentre un FLIP-FLOP sensibile ai fronti di discesa del clock trasferisce in uscita il dato quando il clock “scende” e memorizza lo stato precedente in tutti gli altri casi.

Facendo uso dei segnali CLEAR e PRESET è possibile forzare l’uscita dei circuiti sequenziali a 0 o ad 1 indipendentemente dagli ingressi.

In particolare, il segnale *clear* porta l’uscita a 0, il segnale *preset* porta l’uscita ad 1.

Il segnale *clear* può essere:

- Attivo alto: l’uscita prende il valore 0 se il clear è uguale ad 1;
- Attivo basso: l’uscita prende il valore 0 se il clear è uguale a 0;

Il segnale *preset* può essere:

- Attivo alto: l’uscita prende il valore 1 se il preset è uguale ad 1;
- Attivo basso: l’uscita prende il valore 1 se il preset è uguale a 0;

Questi due segnali possono essere sincroni o asincroni.

Il *clear* sincrono viene anche chiamato *reset*, mentre il *preset* sincrono viene chiamato *set*.

Un insieme di LATCH o un insieme di FLIP-FLOP costituiscono un REGISTRO.

REGISTRO

Per la realizzazione del codice si è fatto uso di registri di flip-flop sensibili al fronte di salita del clock con *clear* attivo basso.

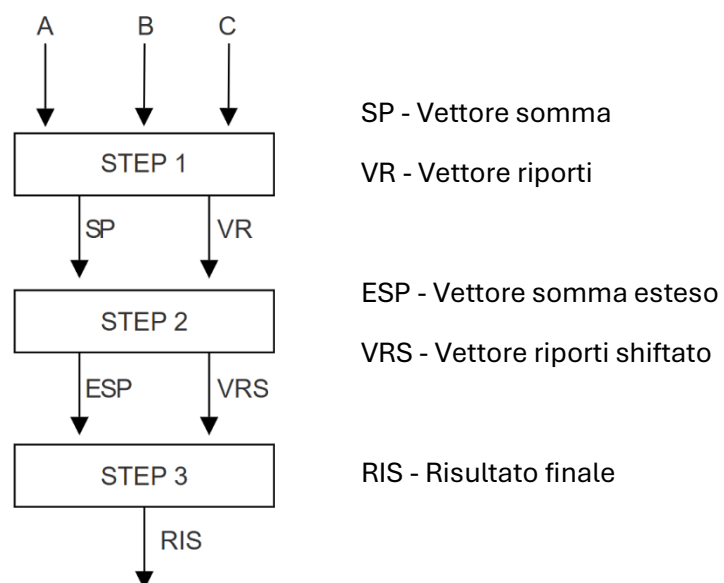
```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3
4  -- Registro provvisto di clear
5  entity Reg is
6      generic (n: integer:= 16);
7      port (D : in std_logic_vector(n-1 downto 0);
8            clk,clr: in std_logic;
9            Q : out std_logic_vector(n-1 downto 0));
10 end Reg;
11
12 --Sensibile ai fronti di salita del clock, con clear asincrono attivo basso
13 architecture Behavioral of Reg is
14
15 begin
16     process (clk,clr) begin
17         if clr = '0' then Q <= (others => '0');
18         elsif rising_edge(clk) then Q <= D;
19         end if;
20     end process;
21 end Behavioral;
```

CARRY-SAVE

Il **Carry-Save** è un sommatore digitale che permette di effettuare la somma fra tre numeri. Esso riceve in input tre operandi e fornisce in output due risultati, ovvero la somma e il riporto.

Il Funzionamento del Carry-Save prevede la divisione dell'operazione di somma in tre passi principali:

- 1) Inizialmente prende le terne di bit che nei tre operandi occupano la stessa posizione e per ciascuna di esse calcola un *bit di somma* e uno di *riporto*;
- 2) A fine primo step il risultato sarà rappresentato da due vettori di bit: il *vettore di somma* e il *vettore dei riporti*. A questo punto bisogna quindi effettuare uno **shift sinistro** del vettore dei riporti. Questo perché, come è possibile intuire, il fine ultimo è quello di effettuare la somma bit a bit dei due vettori, quindi, lo shift è necessario per allineare i riporti alla posizione corretta rispetto al loro contributo nella somma binaria finale;
- 3) Una volta allineati i due vettori il terzo ed ultimo step consiste nell'effettuare una **somma con propagazione di riporto** fra essi, ottenendo il risultato finale della somma;



Quando si effettua lo shift sinistro del vettore dei riporti, bisogna fare attenzione perché i due vettori hanno la stessa dimensione, quindi non saranno più allineati. In particolare, mancherà il bit meno significativo per il vettore dei riporti e il bit più significativo per il vettore di somma.

Per quanto riguarda il vettore dei riporti è sufficiente aggiungere uno '0' come bit meno significativo, mentre per il vettore di somma si distinguono due casi:

- Nel caso in cui si stia lavorando con numeri in *complemento a 2*, bisogna replicare il bit più significativo;
- Nel caso di operandi *unsigned*, basta aggiungere uno '0' come bit più significativo;

Inoltre l'operazione di shift sinistro ha costo 0 dal punto di vista computazionale, perché è possibile gestirlo attraverso i collegamenti col circuito.

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3
4  entity CarrySave is
5      generic (n : integer := 16);
6      port (Op1,Op2,Op3 : in std_logic_vector(2*n-1 downto 0);
7            --Esp = vettore somma esteso, Vrs = Vettore dei riporti shiftato
8            Esp, Vrs : out std_logic_vector(2*n-1 downto 0));
9  end CarrySave;
10
11  architecture Behavioral of CarrySave is
12      signal Vr: std_logic_vector(2*n-1 downto 0);
13
14  begin
15
16      Esp <= ((Op1 xor Op2) xor Op3);
17      Vr <= (Op1 and Op2) or (Op2 and Op3) or (Op1 and Op3);
18      Vrs <= Vr(2*n-2 downto 0) & '0';
19
20  end Behavioral;
```

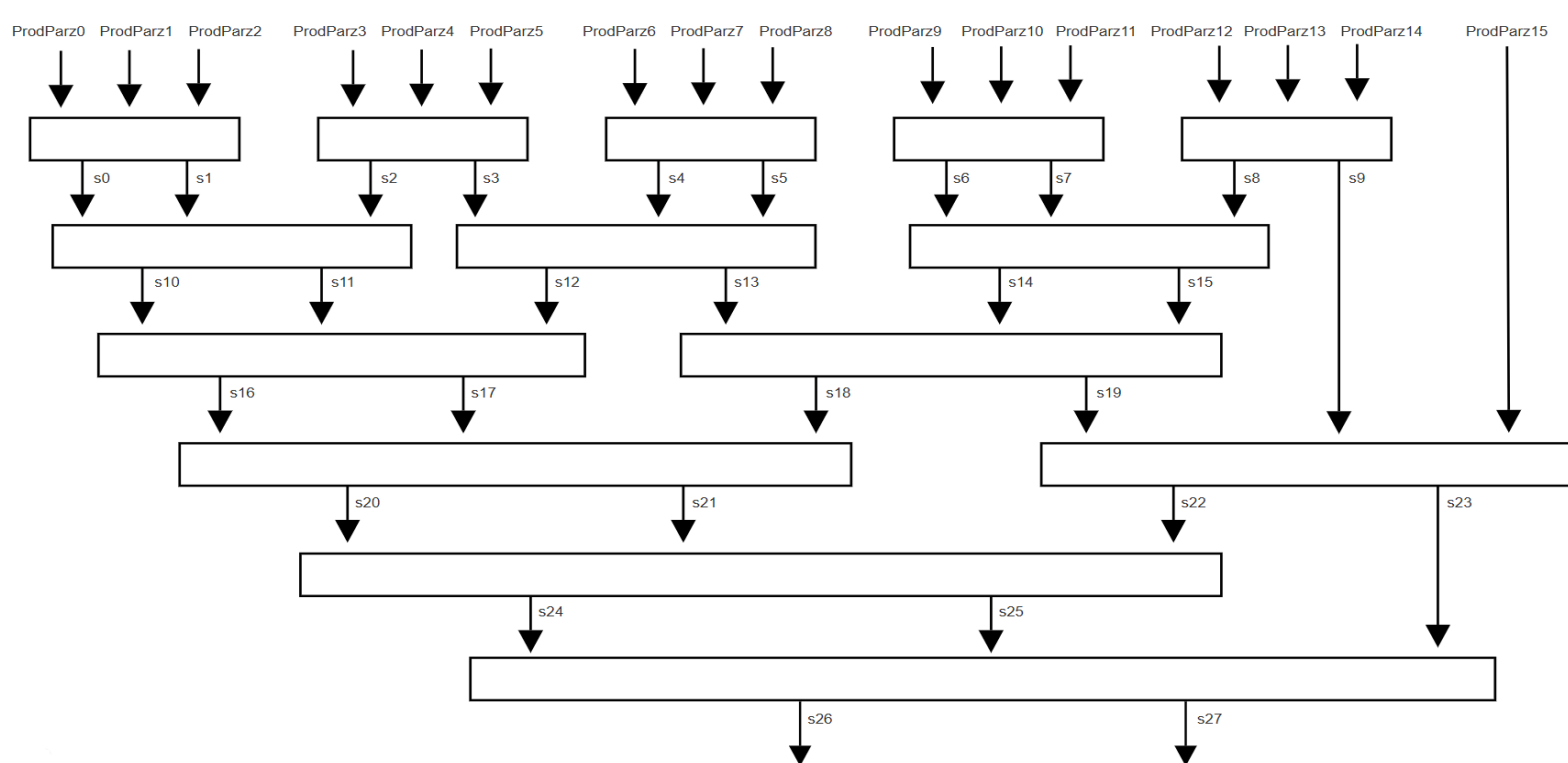
Anche nel caso in cui si debba effettuare la somma fra più di tre operandi, è possibile utilizzare la logica del carry-save.

Ad esempio, se si devono sommare n operandi, si dividono a gruppi da tre, e per ciascuna terna di operandi si applicano gli step uno e due indicati sopra.

Anche se il numero di operandi non è multiplo di tre non è un problema, infatti gli operandi che “restano fuori” si utilizzano nelle iterazioni successive.

A questo punto ogni terna di operandi avrà dato come risultato i due vettori di somma (esteso) e di riporto (shiftato), quindi si riapplicano gli step uno e due, utilizzando come operandi in input, i risultati ottenuti.

Questo procedimento va ripetuto fino a che non si arriva al punto in cui si hanno solo 3 operandi in input, si applicano gli step uno e due per l’ultima volta e a questo punto si procede col terzo step, si effettua quindi la somma con propagazione di riporto.



Per quanto riguarda il **tempo di esecuzione**, si deve considerare che per ogni livello ci sono uno o più blocchi che eseguono gli step uno e due. Come già affermato precedentemente, lo shift non ha alcun costo dal punto di vista computazionale perché può essere gestito con i collegamenti del circuito e la stessa cosa vale per l’estensione. E’ possibile affermare che il ritardo prodotto da ciascun livello (eccetto il livello del sommatore con propagazione di riporto) è di **$T(1)$** .

Per quanto riguarda l’ultimo livello, invece, il ritardo dipende dal numero di bit ricevuti in input.

Teoricamente ciascun livello superiore al sommatore aggiunge un bit agli operandi, ma bisogna tener conto che nel progetto sono stati utilizzati direttamente operandi a **$2 * n = 32$** bit al fine di gestire con semplicità le somme tenendo conto degli shift e dell’estensione dell’ultimo bit.

Questo vuol dire che il ritardo del sommatore con propagazione di riporto sarà proprio $T(2 * n)$.

Quindi, sapendo il tempo di ciascun elemento del circuito, è possibile affermare che:

$$T_{tot} = T(2 * n) + T(1) + T(1) + T(1) + T(1) + T(1) + T(1) \rightarrow T_{tot} = 6T(1) + T(2 * n)$$

MOLTIPLICATORE

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3
4
5  entity Moltiplicatore is
6      generic (n: integer:= 16);
7      port (A, B: in std_logic_vector(n-1 downto 0);
8            UscitaSomma, UscitaCarry: out std_logic_vector(2*n-1 downto 0)
9            );
10 end Moltiplicatore;
11
12
13 architecture Behavioral of Moltiplicatore is
14
15     --Operandi A e B estesi a 32 bit
16     signal extA, extB: std_logic_vector (2*n-1 downto 0):= (others => '0');
17
18     --Le 16 righe dei prodotti parziali
19     signal ProdParz0, ProdParz1, ProdParz2, ProdParz3, ProdParz4, ProdParz5,
20           ProdParz6, ProdParz7, ProdParz8,
21           ProdParz9, ProdParz10, ProdParz11, ProdParz12, ProdParz13, ProdParz14,
22           ProdParz15: std_logic_vector (2*n-1 downto 0):= (others => '0');
23
24     -- Signal per estendere l'i-esimo bit di B e poterlo passare in AND con 'A
25     ,
26     signal estensioneB0, estensioneB1, estensioneB2, estensioneB3,
27           estensioneB4, estensioneB5, estensioneB6,
28           estensioneB7, estensioneB8, estensioneB9, estensioneB10, estensioneB11,
29           estensioneB12, estensioneB13, estensioneB14, estensioneB15:
30           std_logic_vector (n-1 downto 0):= (others => '0');
31
32     --Signal per passare l'output di un carry save a livello precedente in input
33     al carry save del livello successivo
34     signal s0, s1, s2, s3, s4, s5, s6, s7, s8, s9, s10, s11, s12, s13, s14,
35           s15, s16,
36           s17, s18, s19, s20, s21, s22, s23, s24, s25, s26, s27: std_logic_vector (2*n
37           -1 downto 0):= (others => '0');
38
39     component CarrySave is
40         generic (n : integer := 16);
41         port (Op1,Op2,Op3 : in std_logic_vector(2*n-1 downto 0);
42               Esp, Vrs : out std_logic_vector(2*n-1 downto 0)); --Esp = vettore
43               somma esteso, Vrs = Vettore dei riporti shiftato
44     end component;
45
46 begin
47
48
```

```

39 extA(n-1 downto 0) <= A;
40 extB(n-1 downto 0) <= B;
41
42
43
44 --Generico ProdParz
45 --      ProdParz(i) (2*n-1 downto n+i) <= (others => '0');
46 --      ProdParz(i) (n+1-1 downto i) <= estensioneB(i) and extA_;
47 --      ProdParz(i) (i-1 downto 0) <= (others => '0');
48
49 --ProdParz0
50 ProdParz0(2*n-1 downto n+0) <= (others => '0');
51 estensioneB0 <= (others => extB(0));
52 ProdParz0(n+0-1 downto 0) <= extA(n-1 downto 0) and estensioneB0;
53
54 --ProdParz1
55 ProdParz1(2*n-1 downto n+1) <= (others => '0');
56 estensioneB1 <= (others => extB(1));
57 ProdParz1(n+1-1 downto 1) <= extA(n-1 downto 0) and estensioneB1;
58 ProdParz1(1-1 downto 0) <= (others => '0');
59
60 --ProdParz2
61 ProdParz2(2*n-1 downto n+2) <= (others => '0');
62 estensioneB2 <= (others => extB(2));
63 ProdParz2(n+2-1 downto 2) <= extA(n-1 downto 0) and estensioneB2;
64 ProdParz2(2-1 downto 0) <= (others => '0');
65
66 --ProdParz3
67 ProdParz3(2*n-1 downto n+3) <= (others => '0');
68 estensioneB3 <= (others => extB(3));
69 ProdParz3(n+3-1 downto 3) <= extA(n-1 downto 0) and estensioneB3;
70 ProdParz3(3-1 downto 0) <= (others => '0');
71
72 --ProdParz4
73 ProdParz4(2*n-1 downto n+4) <= (others => '0');
74 estensioneB4 <= (others => extB(4));
75 ProdParz4(n+4-1 downto 4) <= extA(n-1 downto 0) and estensioneB4;
76 ProdParz4(4-1 downto 0) <= (others => '0');
77
78 --ProdParz5
79 ProdParz5(2*n-1 downto n+5) <= (others => '0');
80 estensioneB5 <= (others => extB(5));
81 ProdParz5(n+5-1 downto 5) <= extA(n-1 downto 0) and estensioneB5;
82 ProdParz5(5-1 downto 0) <= (others => '0');
83
84 --ProdParz6
85 ProdParz6(2*n-1 downto n+6) <= (others => '0');
86 estensioneB6 <= (others => extB(6));
87 ProdParz6(n+6-1 downto 6) <= extA(n-1 downto 0) and estensioneB6;
88 ProdParz6(6-1 downto 0) <= (others => '0');

```

```

89
90 --ProdParz7
91 ProdParz7(2*n-1 downto n+7) <= (others => '0');
92 estensioneB7 <= (others => extB(7));
93 ProdParz7(n+7-1 downto 7) <= extA(n-1 downto 0) and estensioneB7;
94 ProdParz7(7-1 downto 0) <= (others => '0');
95
96 --ProdParz8
97 ProdParz8(2*n-1 downto n+8) <= (others => '0');
98 estensioneB8 <= (others => extB(8));
99 ProdParz8(n+8-1 downto 8) <= extA(n-1 downto 0) and estensioneB8;
100 ProdParz8(8-1 downto 0) <= (others => '0');
101
102 --ProdParz9
103 ProdParz9(2*n-1 downto n+9) <= (others => '0');
104 estensioneB9 <= (others => extB(9));
105 ProdParz9(n+9-1 downto 9) <= extA(n-1 downto 0) and estensioneB9;
106 ProdParz9(9-1 downto 0) <= (others => '0');
107
108 --ProdParz10
109 ProdParz10(2*n-1 downto n+10) <= (others => '0');
110 estensioneB10 <= (others => extB(10));
111 ProdParz10(n+10-1 downto 10) <= extA(n-1 downto 0) and estensioneB10;
112 ProdParz10(10-1 downto 0) <= (others => '0');
113
114 --ProdParz11
115 ProdParz11(2*n-1 downto n+11) <= (others => '0');
116 estensioneB11 <= (others => extB(11));
117 ProdParz11(n+11-1 downto 11) <= extA(n-1 downto 0) and estensioneB11;
118 ProdParz11(11-1 downto 0) <= (others => '0');
119
120 --ProdParz12
121 ProdParz12(2*n-1 downto n+12) <= (others => '0');
122 estensioneB12 <= (others => extB(12));
123 ProdParz12(n+12-1 downto 12) <= extA(n-1 downto 0) and estensioneB12;
124 ProdParz12(12-1 downto 0) <= (others => '0');
125
126 --ProdParz13
127 ProdParz13(2*n-1 downto n+13) <= (others => '0');
128 estensioneB13 <= (others => extB(13));
129 ProdParz13(n+13-1 downto 13) <= extA(n-1 downto 0) and estensioneB13;
130 ProdParz13(13-1 downto 0) <= (others => '0');
131
132 --ProdParz14
133 ProdParz14(2*n-1 downto n+14) <= (others => '0');
134 estensioneB14 <= (others => extB(14));
135 ProdParz14(n+14-1 downto 14) <= extA(n-1 downto 0) and estensioneB14;
136 ProdParz14(14-1 downto 0) <= (others => '0');
137
138 --ProdParz15

```

```

139 ProdParz15(2*n-1 downto n+15) <= (others => '0');
140 estensioneB15 <= (others => extB(15));
141 ProdParz15(n+15-1 downto 15) <= extA(n-1 downto 0) and estensioneB15;
142 ProdParz15(15-1 downto 0) <= (others => '0');
143
144
145 --Primo livello
146 CS0: CarrySave generic map (16) port map (ProdParz0, ProdParz1, ProdParz2, s0,
147 s1);
148 CS1: CarrySave generic map (16) port map (ProdParz3, ProdParz4, ProdParz5, s2,
149 s3);
150 CS2: CarrySave generic map (16) port map (ProdParz6, ProdParz7, ProdParz8, s4,
151 s5);
152 CS3: CarrySave generic map (16) port map (ProdParz9, ProdParz10, ProdParz11,
153 s6, s7);
154 CS4: CarrySave generic map (16) port map (ProdParz12, ProdParz13, ProdParz14,
155 s8, s9);
156
157 --Secondo livello
158 CS5: CarrySave generic map (16) port map (s0, s1, s2, s10, s11);
159 CS6: CarrySave generic map (16) port map (s3, s4, s5, s12, s13);
160 CS7: CarrySave generic map (16) port map (s6, s7, s8, s14, s15);
161
162 --Terzo livello
163 CS8: CarrySave generic map (16) port map (s10, s11, s12, s16, s17);
164 CS9: CarrySave generic map (16) port map (s13, s14, s15, s18, s19);
165
166 --Quarto livello
167 CS10: CarrySave generic map (16) port map (s16, s17, s18, s20, s21);
168 CS11: CarrySave generic map (16) port map (s19, s9, ProdParz15, s22, s23);
169
170 --Quinto livello
171 CS12: CarrySave generic map (16) port map (s20, s21, s22, s24, s25);
172
173 --Sesto livello
174 CS13: CarrySave generic map (16) port map (s24, s25, s23, s26, s27);
175
176 UscitaSomma<= s26;
177 UscitaCarry<= s27;
178 end Behavioral;

```

ADDER CON PROPAGAZIONE DI RIPORTO

Per poter completare la logica del carry save, è necessario andare a sommare i due vettori ottenuti dal component Moltiplicatore propagando il riporto. Si è dunque definito un semplice sommatore, che assolvesse a questo compito:

```
1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3 use IEEE.NUMERIC_STD.ALL;
4
5 entity Adder is
6     generic (n:integer:= 16);
7     Port (
8         vect_somma, vect_carry : in std_logic_vector(n-1 downto 0);
9         Sum: out std_logic_vector(n downto 0)
10    );
11 end Adder;
12
13 architecture Behavioral of Adder is
14
15     signal p,g: std_logic_vector (n downto 0);
16     signal carry: std_logic_vector (n+1 downto 0);
17 begin
18
19     carry(0) <= '0';
20     p <= '0' & (vect_somma xor vect_carry);
21     g <= '0' & (vect_somma and vect_carry);
22
23     carry (n+1 downto 1) <= g or (p and carry (n downto 0) );
24
25     Sum <= p xor carry(n downto 0);
26
27 end Behavioral;
```

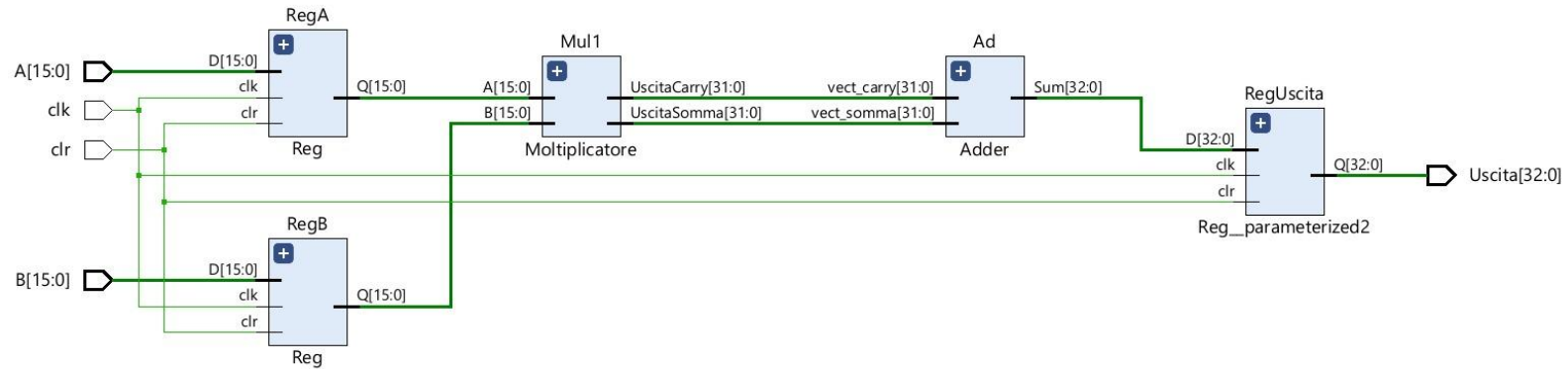
MODULO TOP

Dopo aver analizzato tutte le sotto-componenti, è ora possibile istanziarle utilizzando i segnali appropriati per collegarle tra loro.

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3
4  entity Top is
5      generic(n:integer:= 16);
6      port(A,B: in std_logic_vector(n-1 downto 0);
7          clk,clr: in std_logic;
8          Uscita: out std_logic_vector(2*n downto 0));
9  end Top;
10
11  architecture Behavioral of Top is
12
13      component Reg is
14          generic (n: integer:= 16);
15          port (D : in std_logic_vector(n-1 downto 0);
16              clk,clr: in std_logic;
17              Q : out std_logic_vector(n-1 downto 0));
18      end component;
19
20      component Moltiplicatore is
21          generic (n: integer:= 16);
22          port (A, B: in std_logic_vector(n-1 downto 0);
23              UscitaSomma, UscitaCarry: out std_logic_vector(2*n-1 downto 0)
24              );
25      end component;
26
27      component Adder is
28          generic (n:integer:= 32);
29          Port (
30              vect_somma, vect_carry : in std_logic_vector(n-1 downto 0);
31              Sum: out std_logic_vector(n downto 0)
32          );
33      end component;
34
35      signal Ra, Rb: std_logic_vector(n-1 downto 0); -- out registri
36      signal sum,carry: std_logic_vector(2*n-1 downto 0); --out operazione mul
37      signal sommaFinale: std_logic_vector(2*n downto 0); --out operazione somma
38          con riporto
39
40  begin
41
42      RegA: Reg generic map (16) port map (A, clk, clr, Ra);
43      RegB: Reg generic map (16) port map (B, clk, clr, Rb);
44      Mul1: Moltiplicatore generic map(16) port map (Ra,Rb,sum,carry);
45      Ad: Adder generic map(32) port map(sum,carry,sommaFinale);
46      RegUscita: Reg generic map(33) port map (sommaFinale, clk, clr, Uscita);
47  end Behavioral;
```

SCHEMATIC

Volendo offrire una panoramica generale e completa del circuito, si riporta anche lo schematic, utile per la visualizzazione dei signal tra i vari sottocomponenti:



TESTBENCH

Tenendo in considerazione i precedenti codici VHDL, è stata effettuata la simulazione behavioral, in modo da verificare il corretto funzionamento del circuito per gli ingressi specificati.

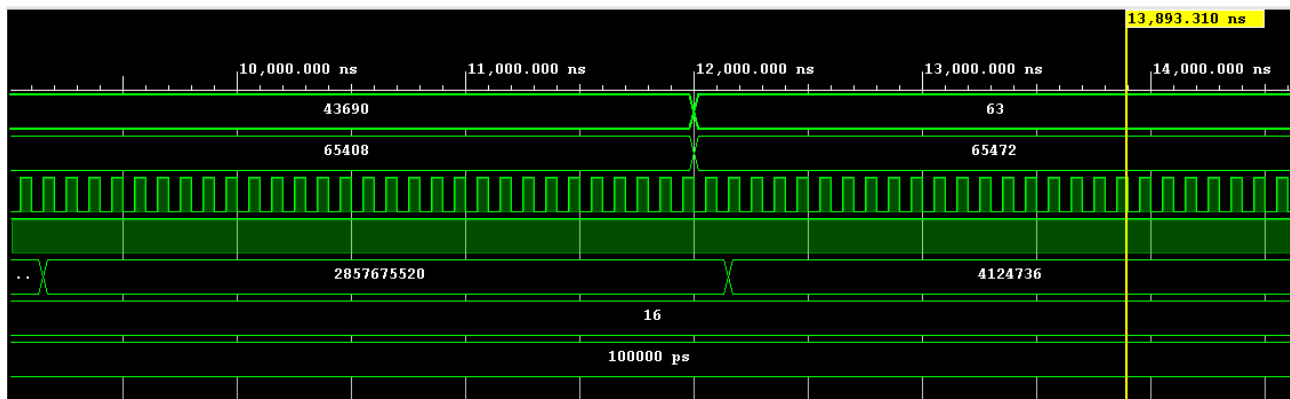
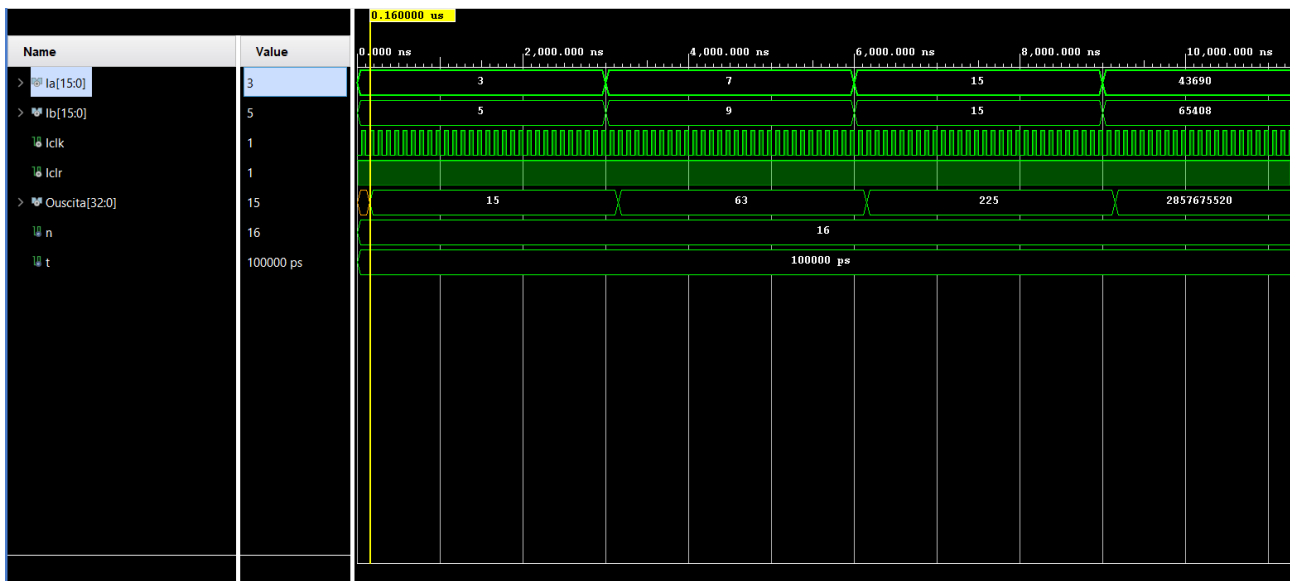
```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3
4  entity Testbench_top is
5      generic (n: integer:=16);
6  end Testbench_top;
7
8  architecture Behavioral of Testbench_top is
9
10     component Top is
11         generic(n:integer:= 16);
12         port(A,B: in std_logic_vector(n-1 downto 0);
13             clk,clr: in std_logic;
14             Uscita: out std_logic_vector(2*n downto 0));
15     end component;
16
17     signal Ia, Ib: std_logic_vector (n-1 downto 0) := (others => '0');
18     signal Iclk: std_logic := '0';
19     signal Iclr : std_logic := '1';
20
21     signal Ouscita: std_logic_vector(2*n downto 0);
22
23     constant t: time := 100 ns;
24
25 begin
26     CUT: Top port map (Ia, Ib, Iclk, Iclr, Ouscita);
27
28     process begin
29         wait for t/2;
30         Iclk <= not Iclk;
31     end process;
32
33     process begin
34
35         Ia <= "0000000000000011";
36         Ib <= "0000000000000101";
37         wait for 30 * t;
38
39
40         Ia <= "0000000000000111";
41         Ib <= "0000000000001001";
42         wait for 30 * t;
43
44         Ia <= "0000000000001111";
45         Ib <= "0000000000001111";
46         wait for 30 * t;
47
48
```

```

49      Ia <= "1010101010101010";
50      Ib <= "1111111110000000";
51      wait for 30 * t;
52
53      Ia <= "0000000000111111";
54      Ib <= "1111111110000000";
55      wait for 30 * t;
56
57  end process;
58 end Behavioral;

```

Si visualizzano i risultati prodotti:



SINTESI

Dissipazione di potenza

Power estimation from Synthesized netlist. Activity derived from constraints files, simulation files or vectorless analysis. Note: these early estimates can change after implementation.

Total On-Chip Power: **39.224 W (Junction temp exceeded!)**

Design Power Budget: **Not Specified**

Power Budget Margin: **N/A**

Junction Temperature: **125,0°C**

Thermal Margin: **-377,4°C (-32,1 W)**

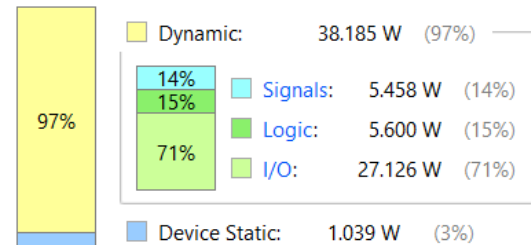
Effective θ_{JA} : 11,5°C/W

Power supplied to off-chip devices: 0 W

Confidence level: **Low**

[Launch Power Constraint Advisor](#) to find and fix invalid switching activity

On-Chip Power



In rosso viene segnalato che la temperatura di giunzione supera il limite massimo raccomandato.

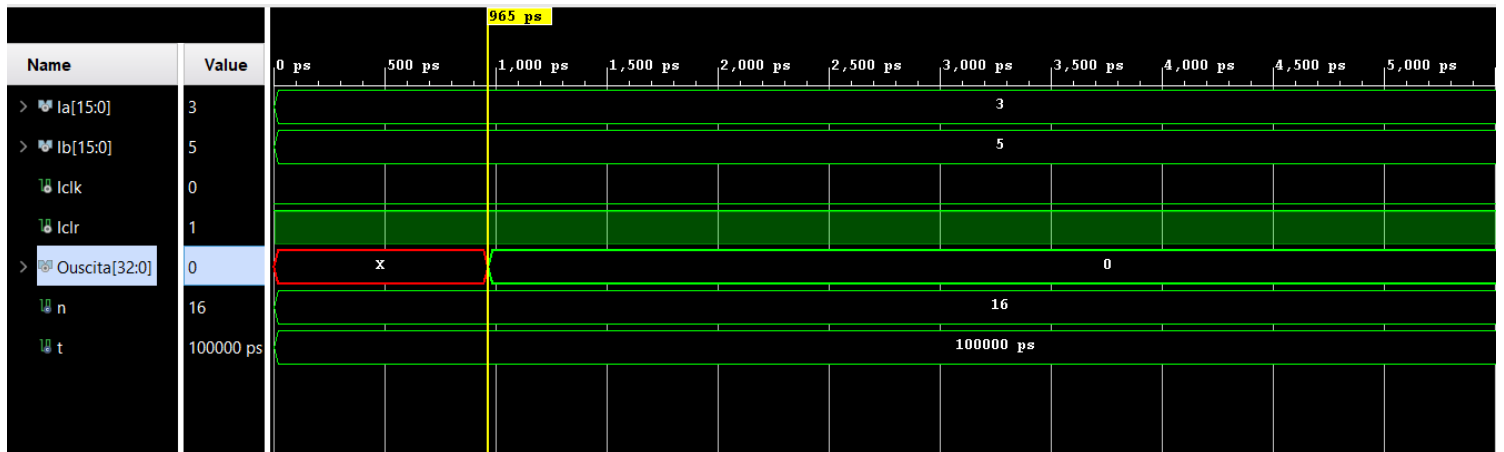
Report sull'utilizzazione delle risorse

28	1. Slice Logic
29	-----
30	
31	+-----+-----+-----+-----+-----+-----+
32	Site Type Used Fixed Prohibited Available Util%
33	+-----+-----+-----+-----+-----+-----+
34	Slice LUTs* 393 0 0 53200 0.74
35	LUT as Logic 393 0 0 53200 0.74
36	LUT as Memory 0 0 0 17400 0.00
37	Slice Registers 65 0 0 106400 0.06
38	Register as Flip Flop 65 0 0 106400 0.06
39	Register as Latch 0 0 0 106400 0.00
40	F7 Muxes 0 0 0 26600 0.00
41	F8 Muxes 0 0 0 13300 0.00
42	+-----+-----+-----+-----+-----+-----+

La tabella mostra che nel progetto sono state utilizzate 393 LUTs e 65 registri.

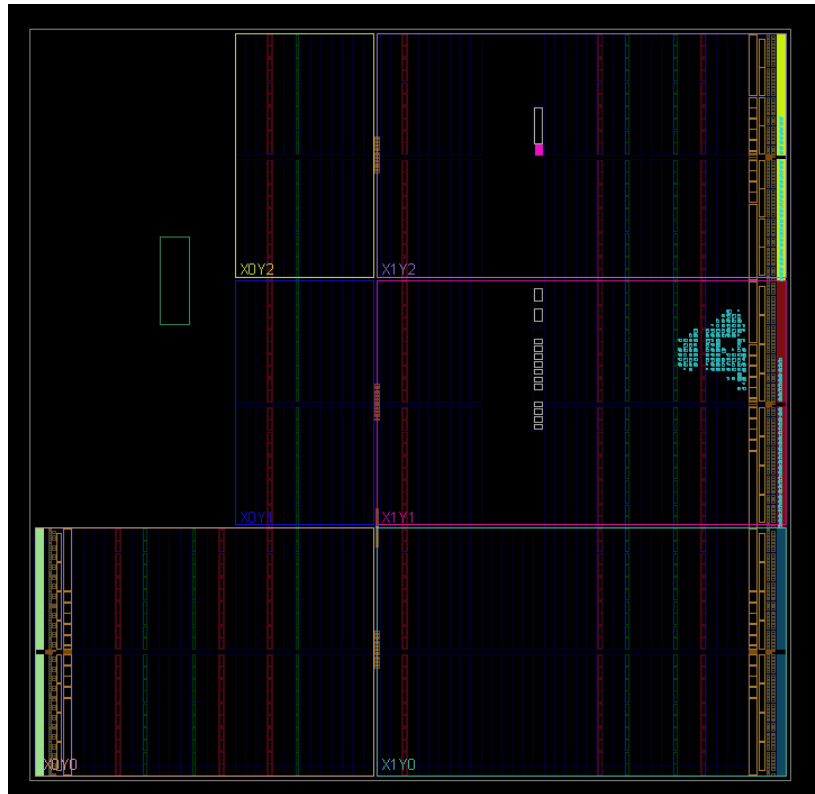
SIMULAZIONE POST-SINTESI

Nella simulazione post-sintesi si può notare la presenza di un breve periodo iniziale in cui l'uscita assume un valore indeterminato (X), dovuto al ritardo delle porte logiche, prima che i segnali possano stabilizzarsi.



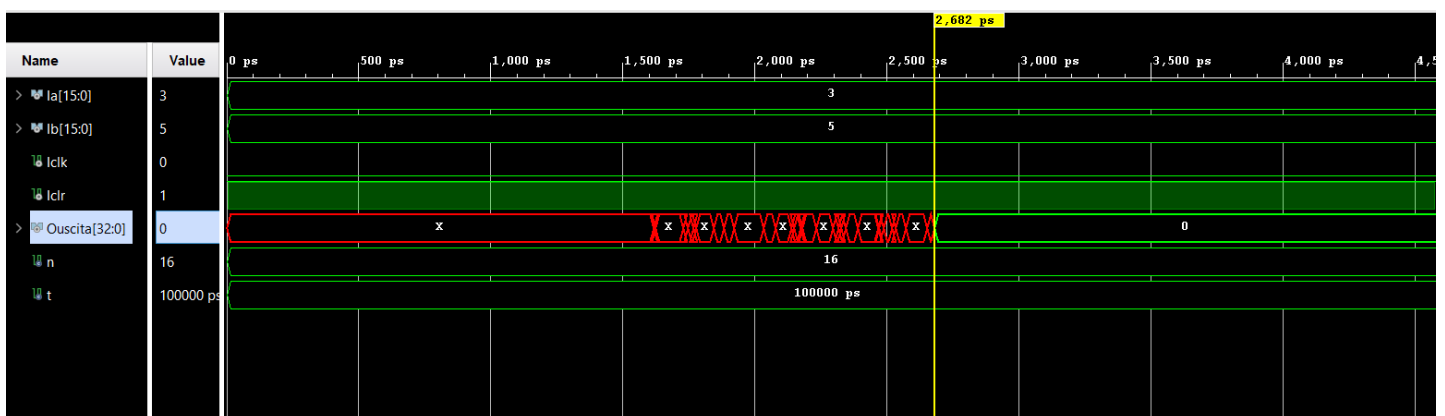
IMPLEMENTAZIONE

Rappresentazione del layout fisico del design implementato sul dispositivo, di cui sono evidenziate in azzurro le parti utilizzate.



SIMULAZIONE POST-IMPLEMENTATION

Nella simulazione post-implementation sono visibili ritardi maggiori rispetto alla simulazione post-sintesi, dovuti all'implementazione del circuito su device.



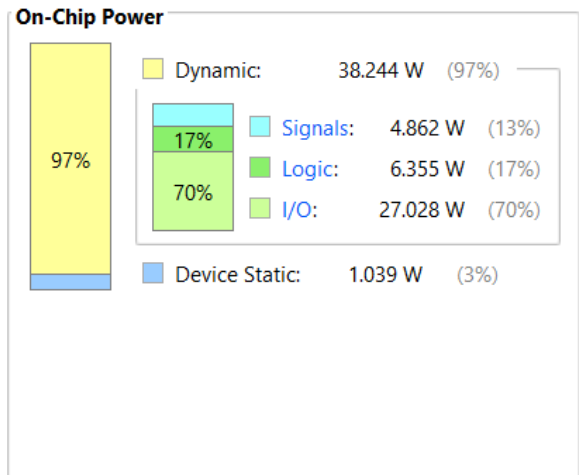
AGGIUNTA DI UN CONSTRAINT

Quando il circuito non presenta vincoli in termini di frequenza, il risultato dell'implementazione segnala un malfunzionamento. In particolare, la temperatura di giunzione all'interno dei dispositivi supera quella massima consentita:

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power: **39.284 W (Junction temp exceeded!)**
Design Power Budget: **Not Specified**
Power Budget Margin: **N/A**
Junction Temperature: **125,0°C**
Thermal Margin: -378,1°C (-32,2 W)
Effective θ_{JA} : 11,5°C/W
Power supplied to off-chip devices: 0 W
Confidence level: **Low**

[Launch Power Constraint Advisor](#) to find and fix invalid switching activity



Inoltre, la dissipazione di potenza dinamica supera notevolmente quella di potenza statica.

Per far lavorare il circuito ad una certa frequenza, introduciamo un constraint sul segnale di clock.

Il constraint fissato nel circuito prevede un periodo di clock di 10 ns.

```
1 | create clock -period 10.000 -name myClock -waveform {0.000 5.000} [get nets clk]
```

Con la presenza del constraint, i tempi di esecuzione della sintesi e dell'implementazione sono leggermente dilatati, in quanto Vivado deve rispettare il vincolo temporale impostogli.

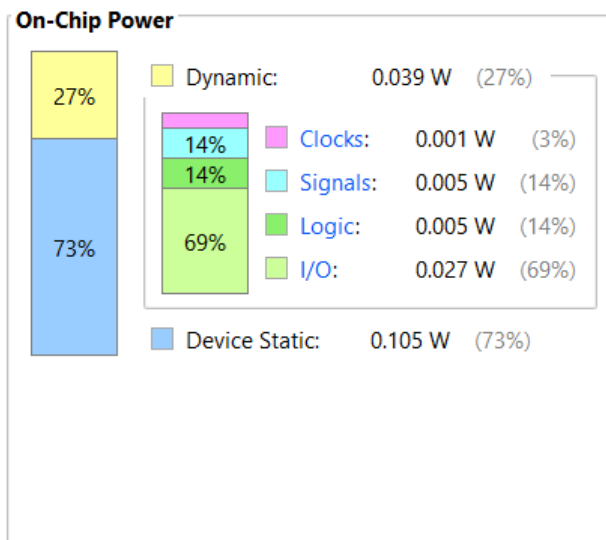
Dall'analisi dei risultati relativi all'implementazione, si può notare che nel report sulla potenza è apparsa l'indicazione sulla dissipazione di potenza da parte del segnale di clock.

Non è più presente invece la segnalazione sulla temperatura di giunzione, poiché la dissipazione di potenza è stata stimata alla frequenza di funzionamento che corrisponde ai 10 ns definiti nel constraint.

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

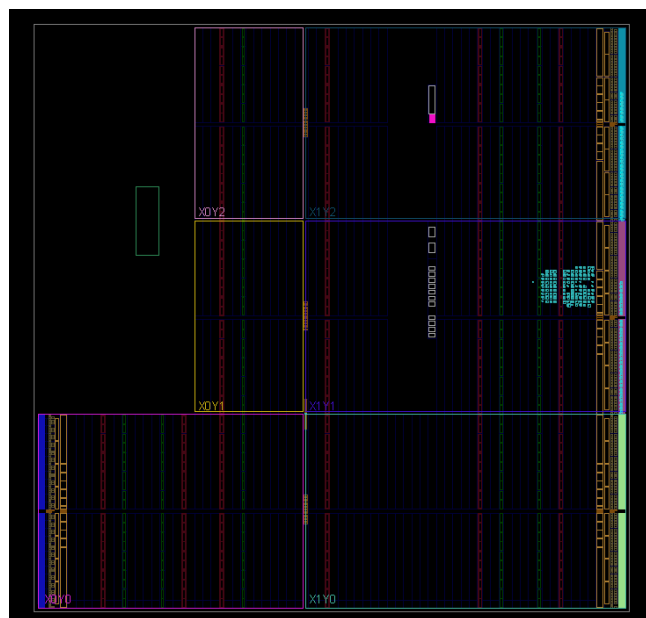
Total On-Chip Power: 0.144 W
Design Power Budget: Not Specified
Power Budget Margin: N/A
Junction Temperature: 26,7°C
Thermal Margin: 73,3°C (6,0 W)
Effective θ_{JA} : 11,5°C/W
Power supplied to off-chip devices: 0 W
Confidence level: Low

[Launch Power Constraint Advisor](#) to find and fix invalid switching activity



La diminuzione della temperatura di giunzione è dovuta proprio al fatto che, adesso, la dissipazione di potenza statica è nettamente superiore a quella di potenza dinamica, quindi il dispositivo viene riscaldato di meno. La presenza del constraint consente dunque di ottimizzare il design e ridurre considerevolmente il consumo di potenza dinamica. Ciò vuol dire che ci sono molte più risorse che dissipano staticamente rispetto a quelle che dissipano dinamicamente.

Questo è evidente anche dall'analisi del dispositivo: le risorse effettivamente utilizzate nel circuito sono quelle evidenziate in azzurro, e costituiscono solo una piccola parte rispetto all'intero dispositivo.



Ulteriori informazioni di notevole interesse si possono rintracciare nel *Report Timing Summary*, nel quale sono descritte tutte le caratteristiche temporali del circuito relative al constraint fissato.

In questa sezione, Vivado ci dice se il constraint specificato è stato rispettato.

Il valore indicato da WNS (Worst Negative Slack), se positivo, indica che il circuito potrebbe lavorare ad una frequenza superiore rispetto a quella impostata; se negativo, indica la presenza di almeno un percorso nel design che non rispetta i requisiti di temporizzazione.

Design Timing Summary

WNS (ns)	TNS (ns)	TNS Failing Endpoints	TNS Total Endpoints	WHS (ns)	THS (ns)	THS Failing Endpoints	THS Total Endpoints	WPWS (ns)	TPWS (ns)
0.543	0.000	0	33	0.432	0.000	0	33	4.500	0.000

All user specified timing constraints are met.

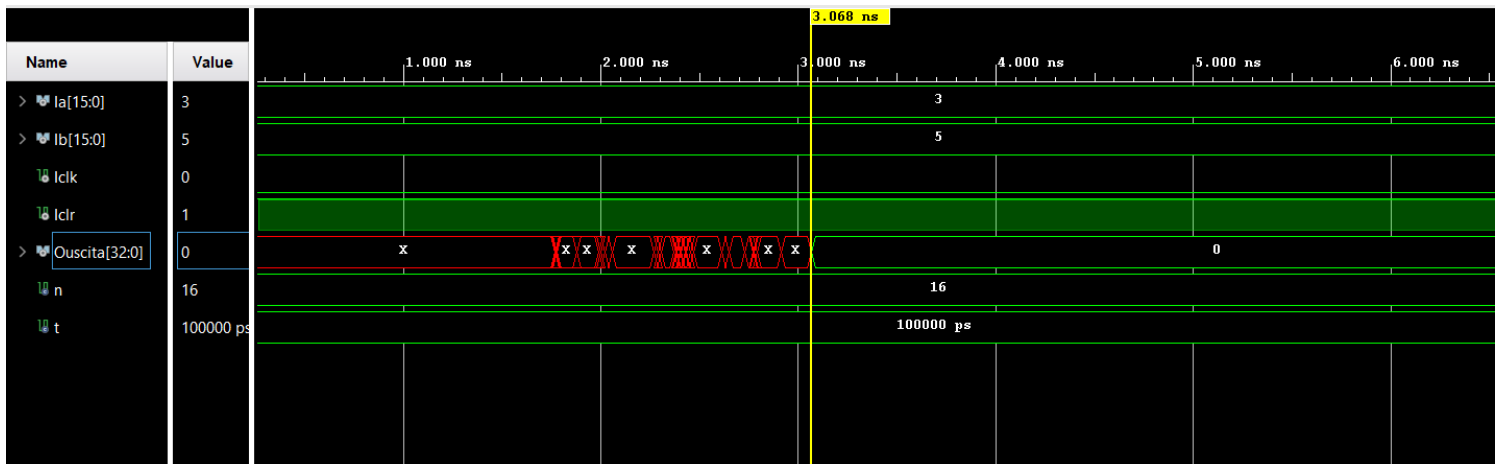
In questo caso, il constraint di timing è stato soddisfatto. Il valore di WNS indica che il circuito potrebbe lavorare ancora più velocemente, cioè ai 10 ns impostati potremmo sottrarre 0,543 ns.

Al constraint di 10 ns, corrisponde una frequenza di 100 MHz.

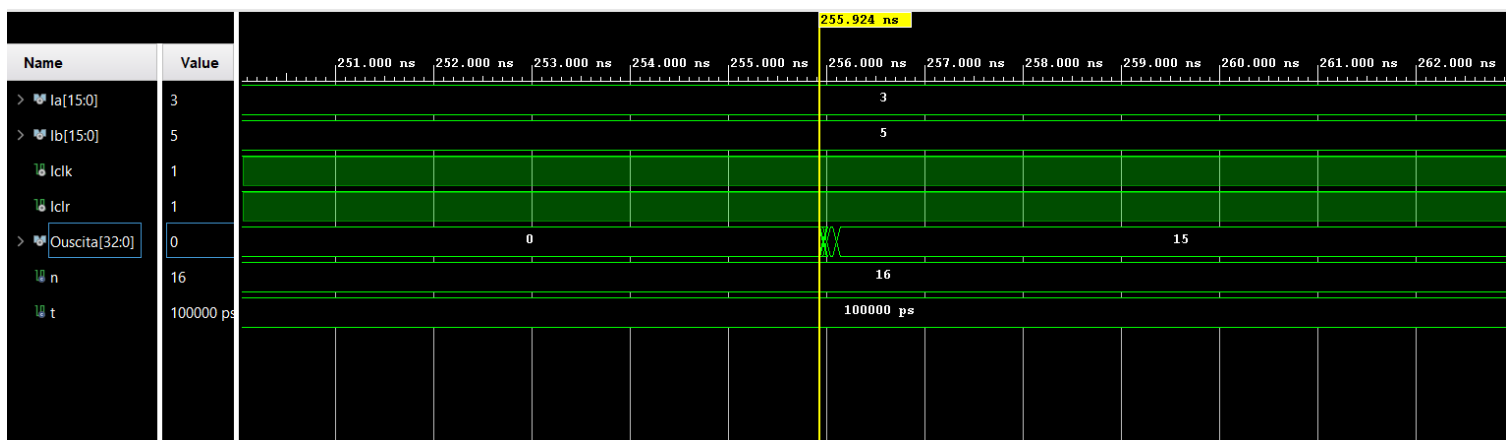
Clock Summary

Clock	Waveform(ns)	Period(ns)	Frequency(MHz)
myClock	{0.000 5.000}	10.000	100.000

Eseguendo la simulazione post-implementation, si nota la presenza di ritardi dovuti alla logica del circuito ed alle interconnessioni:



Inoltre, a seguito dell'attivazione automatica di un segnale di global reset, i registri vengono bloccati e l'uscita in una fase iniziale rimane a 0.



Per tenere conto di ciò, si potrebbe aggiungere nel testbench un'attesa che ritarda l'avvio delle operazioni.